

Hardware-Aware Dynamic Batch Compaction for Early-Exit Cross-Encoder Re-Rankers

Shayan Nasiri

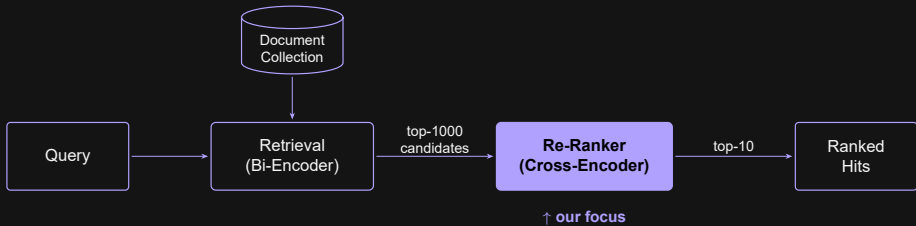
Winter 2026

CSC 2210 — University of Toronto

[Recording on SharePoint](#)



Background on RAGs



- **Retrieval** (bi-encoder): fast, shallow match over millions of docs
- **Re-ranking** (cross-encoder): accurate but quadratic cost
- 🔭 Our target: the re-ranker — RAG's dominant latency cost

Make early-exit cross-encoders actually fast on GPUs, without sacrificing ranking accuracy.

1.  **Systems goal** — design a custom GPU kernel that physically compacts the batch between transformer layers, so exited documents stop consuming compute
2.  **Accuracy goal** — stay within $\leq 5\%$ relative MRR@10 of an unmodified cross-encoder baseline
3.  If the target isn't met, analyzing why and where it breaks is itself the contribution



What's been done:

- Early-exit classifiers:
BranchyNet, DeeBERT, FastBERT
- Patience & distillation:
PABEE, RomeBERT, LeeBERT
- Calibrated exit signals:
DE3-BERT, COSEE, BEEM
- IR-specific pruning:
E2Rank, SEE



What's missing:

- No physical batch-compaction kernel for cross-encoder IR
- Prior speedups measured on classification (GLUE), not ranking metrics like MRR
- Exit criteria always a single scalar threshold, not per-ramp



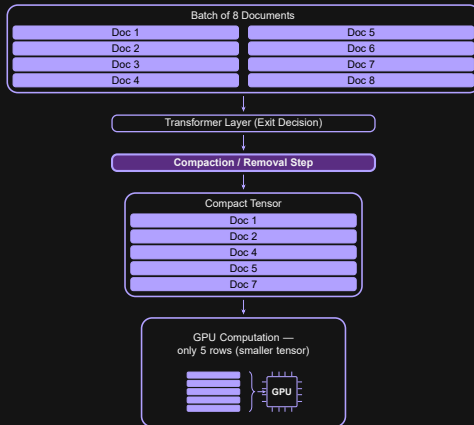
The Jagged-Batch Problem

Naive Early Exit (No Speedup)



3 fewer rows
less GPU work

Triton Compaction (Real Speedup)





Our Approach

1. Triton compaction kernel

Physically shrink the batch between layers

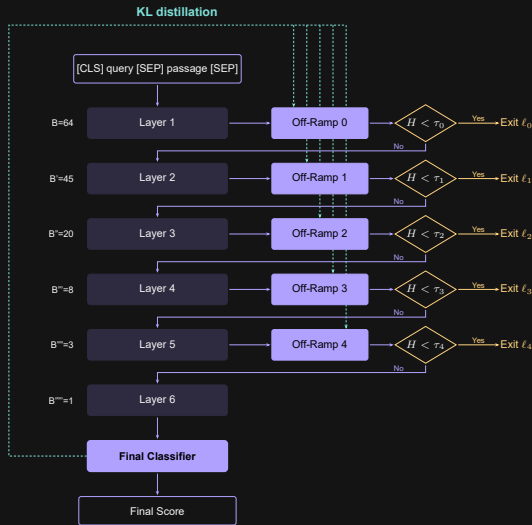
2. Joint training + KL distillation

Off-ramps learn from the final classifier

3. Per-ramp threshold vector

$\tau \in \mathbb{R}^5$, one per off-ramp

$7^5=16,807$ -point grid search



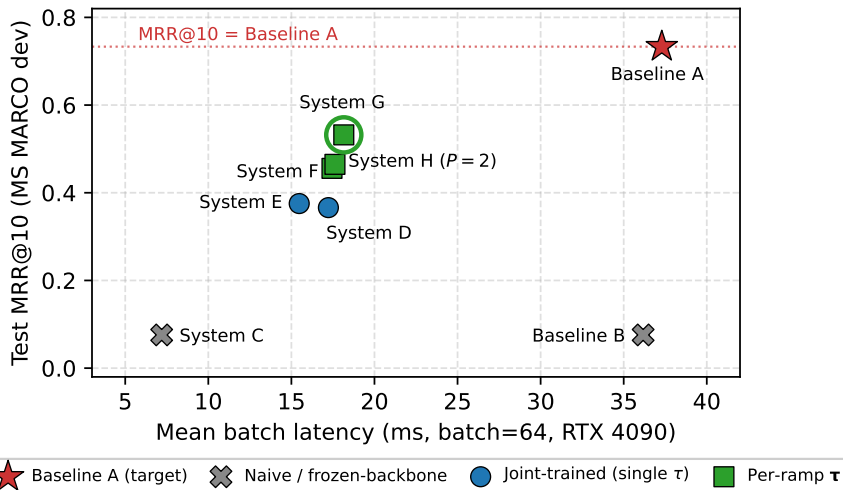
Eight systems stacked cumulatively

System	What changes	MRR@10	Lat. (ms)
Baseline A	full cross-encoder (target)	0.733	37.3
Baseline B	+ naive early exit (jagged batch)	0.076	36.2
System C	+ Triton compaction (frozen backbone)	0.076	7.2
System D	+ joint training	0.366	17.2
System E	+ KL self-distillation	0.375	15.5
System F	+ per-ramp τ on D weights	0.456	17.4
 System G	+ per-ramp τ on E weights	0.532	18.2
System H	+ PABEE patience $P=2$	0.465	17.6

Each row adds one idea on top of the previous. **System G**: $2.05\times$ speedup, 27% below Baseline A.



Results



Takeaways

✓ What worked

- **Triton compaction** isolates the hardware win — $5\times$ speedup at matched MRR; kernel is accuracy-neutral
- **Joint training + KL distillation** recover MRR from $0.076 \rightarrow 0.375$
- **Per-ramp $\tau \times \text{KL}$ = super-additive** — System G: 0.53 MRR at $2.05\times$ speedup over Baseline A

✗ What didn't

- Missed the $\leq 5\%$ target — System G is 27% below Baseline A; the remaining gap is an **off-ramp representation problem**, not kernel or threshold
- **PABEE patience is a clean negative** — $P=2$ forces every document through at least 2 ramps, adding latency without recovering MRR

Thank you! 🙌

Code: https://github.com/ShayanNasiri/csc2210_project